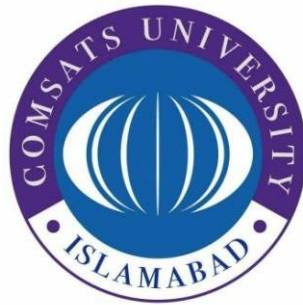


# *COMSATS University Islamabad*

## *Attock Campus*



### *Department Of Computer Science*

|                       |                                 |
|-----------------------|---------------------------------|
| <i>Course</i>         | <i>Information Security Lab</i> |
| <i>Instructor</i>     | <i>Ambreen Gul</i>              |
| <i>Assignment No.</i> | <i>03</i>                       |

|                         |                    |
|-------------------------|--------------------|
| <i>Registration No.</i> | <i>Name</i>        |
| <i>FA24-BSE-043</i>     | <i>Sohaib Wali</i> |

# Assignment 3

## Task 1: Basic Blockchain Implementation

```
1  import hashlib
2  import time
3
4
5  class Block: 2 usages
6      def __init__(self, index, data, previous_hash):
7          self.index = index
8          self.timestamp = time.strftime("%Y-%m-%d %H:%M:%S")
9          self.data = data
10         self.previous_hash = previous_hash
11         self.hash = self.calculate_hash()
12
13     def calculate_hash(self): 2 usages
14         content = f"{self.index}{self.timestamp}{self.data}{self.previous_hash}"
15         return hashlib.sha256(content.encode()).hexdigest() # generate hash
16
17
18 # ----- Blockchain Class -----
19 class Blockchain: 1 usage
20     def __init__(self):
21         self.chain = [self.create_genesis_block()] # start with first block
22
23     def create_genesis_block(self): 1 usage
24         return Block(index=0, data="Genesis Block", previous_hash="0") # first block
25
26     def add_block(self, data): 4 usages
27         last_block = self.chain[-1] # get last block
28         new_block = Block(len(self.chain), data, last_block.hash) # create new block
29         self.chain.append(new_block) # add to chain
30
31     def display_chain(self): 2 usages
32         for block in self.chain:
33             print("Index:", block.index)
34             print("Timestamp:", block.timestamp)
35             print("Data:", block.data)
36             print("Previous Hash:", block.previous_hash)
37             print("Hash:", block.hash)
38             print()
39
```

```

38         print()
39
40     def is_valid(self): 2 usages
41         for i in range(1, len(self.chain)):
42             curr = self.chain[i]
43             prev = self.chain[i - 1]
44
45             if curr.hash != curr.calculate_hash(): # check hash
46                 return False
47
48             if curr.previous_hash != prev.hash: # check link
49                 return False
50
51         return True
52
53
54
55 bc = Blockchain() # create blockchain
56
57 # add blocks
58 bc.add_block("Alice pays Bob $50")
59 bc.add_block("Bob pays Charlie $30")
60 bc.add_block("Charlie pays Dave $20")
61 bc.add_block("Dave pays Eve $10")
62
63 # show blockchain
64 bc.display_chain()
65
66 # check validity
67 print("Valid:", bc.is_valid())
68
69 #Tampering
70 bc.chain[2].data = "Bob pays Charlie $9999" # change data
71
72 # check again
73 print("Valid after tampering:", bc.is_valid())
74
75 # show chain again
76 bc.display_chain()

```

**Output:**

↓  
≡  
±  
🖨  
🗑

Index: 0  
Timestamp: 2026-05-03 12:35:16  
Data: Genesis Block  
Previous Hash: 0  
Hash: 42db0c141f63374a8dd4ba9a53d47c6b4eb0b4e07d6d79c5b7029f148960e188

Index: 1  
Timestamp: 2026-05-03 12:35:16  
Data: Alice pays Bob \$50  
Previous Hash: 42db0c141f63374a8dd4ba9a53d47c6b4eb0b4e07d6d79c5b7029f148960e188  
Hash: 7ffe46842ae858f6456ff4f1b1de892e3a8e8c68c5d4ee34102d830a70ff0d40

Index: 2  
Timestamp: 2026-05-03 12:35:16  
Data: Bob pays Charlie \$30  
Previous Hash: 7ffe46842ae858f6456ff4f1b1de892e3a8e8c68c5d4ee34102d830a70ff0d40  
Hash: 36e48be8cb78dedfc9253802a34bbfeac338f73545f030fdbcbf079c3d1dcea5

Index: 3  
Timestamp: 2026-05-03 12:35:16  
Data: Charlie pays Dave \$20  
Previous Hash: 36e48be8cb78dedfc9253802a34bbfeac338f73545f030fdbcbf079c3d1dcea5  
Hash: aad1dc391756d65020affe2896b88261f70484436ae7f90b2fb3e45877d914c4

Index: 4  
Timestamp: 2026-05-03 12:35:16  
Data: Dave pays Eve \$10  
Previous Hash: aad1dc391756d65020affe2896b88261f70484436ae7f90b2fb3e45877d914c4  
Hash: e299ef0464521b5613307511e87c3a4da353ceef70ccc8f609a5062f70dce4c3

Valid: True  
Valid after tampering: False  
Index: 0  
Timestamp: 2026-05-03 12:35:16  
Data: Genesis Block  
Previous Hash: 0  
Hash: 42db0c141f63374a8dd4ba9a53d47c6b4eb0b4e07d6d79c5b7029f148960e188

Index: 1  
Timestamp: 2026-05-03 12:35:16  
Data: Alice pays Bob \$50  
Previous Hash: 42db0c141f63374a8dd4ba9a53d47c6b4eb0b4e07d6d79c5b7029f148960e188  
Hash: 7ffe46842ae858f6456ff4f1b1de892e3a8e8c68c5d4ee34102d830a70ff0d40

Index: 2  
Timestamp: 2026-05-03 12:35:16  
Data: Bob pays Charlie \$9999  
Previous Hash: 7ffe46842ae858f6456ff4f1b1de892e3a8e8c68c5d4ee34102d830a70ff0d40  
Hash: 36e48be8cb78dedfc9253802a34bbfeac338f73545f030fdbcbf079c3d1dcea5

Index: 3  
Timestamp: 2026-05-03 12:35:16  
Data: Charlie pays Dave \$20  
Previous Hash: 36e48be8cb78dedfc9253802a34bbfeac338f73545f030fdbcbf079c3d1dcea5  
Hash: aad1dc391756d65020affe2896b88261f70484436ae7f90b2fb3e45877d914c4

Index: 4  
Timestamp: 2026-05-03 12:35:16  
Data: Dave pays Eve \$10  
Previous Hash: aad1dc391756d65020affe2896b88261f70484436ae7f90b2fb3e45877d914c4  
Hash: e299ef0464521b5613307511e87c3a4da353ceef70ccc8f609a5062f70dce4c3

## Block Structure

Each block contains:

- Index which tells position of block in chain
- Timestamp: time of creation
- Data: transaction/message
- Previous Hash link to previous block
- Hash which gives unique digital fingerprint

The hash is created using SHA-256, making it secure and unique.

## Hashing Mechanism

`hashlib.sha256(content.encode()).hexdigest()`

This converts block data into a fixed unique string

Even a small change in data completely different hash

This is what makes blockchain tamper-resistant

## Genesis Block (First Block)

`create_genesis_block()`

- First block in the chain
- Has no previous block
- Previous hash is set to "0"

It acts as the starting point of blockchain

## Adding New Blocks

When a new block is added:

- System takes the last block
- Uses its hash as previous hash
- Creates a new block
- Appends it to the chain

This creates a linked structure (chain of blocks)

## **Security Analysis of Blockchain**

### **Data Integrity**

There is a hash in each block.

When data is modified -> hash is modified.

### **Linking Blocks**

Past hash is stored in each block. This provides a safe chain.

### **Tampering Detection**

When one block is modified → chain is invalid. It has been proved by using `is_valid()` function.

### **Use of SHA-256**

Strong cryptographic hashing

Almost impossible to reverse

## **Task 2: Elliptic Curve Cryptography (ECC) Basics**

```
1  p = 17
2  a = 2
3  b = 2
4  # Modular Inverse
5  def mod_inv(x, p): 2 usages
6      return pow(x, p - 2, p) # Fermat's little theorem
7
8
9  def point_add(P, Q): 4 usages
10     if P == "0":
11         return Q
12     if Q == "0":
13         return P
14
15     x1, y1 = P
16     x2, y2 = Q
17
18     if P != Q:
19         m = (y2 - y1) * mod_inv(x2 - x1, p)
20     else:
21         m = (3 * x1 * x1 + a) * mod_inv(2 * y1, p)
22
23     m = m % p
24
25     x3 = (m * m - x1 - x2) % p
26     y3 = (m * (x1 - x3) - y1) % p
27
28     return (x3, y3)
29
30 # Scalar Multiplication
31 def scalar_mult(k, P): 4 usages
32     result = "0"
33     temp = P
34
35     while k > 0:
36         if k % 2 == 1:
37             result = point_add(result, temp)
38             temp = point_add(temp, temp)
39             k = k // 2
```



```

        K = K // 2

    return result

G = (5, 1)

# Key Generation
private_key = 7
public_key = scalar_mult(private_key, G)

# Encryption
def encrypt(M, public_key): 1 usage
    k = 3
    C1 = scalar_mult(k, G)
    C2 = point_add(M, scalar_mult(k, public_key))
    return C1, C2

# Decryption
def decrypt(C1, C2, private_key): 1 usage
    S = scalar_mult(private_key, C1)
    minus_S = (S[0], (-S[1]) % p)
    return point_add(C2, minus_S)

M = (6, 3)

C1, C2 = encrypt(M, public_key)
print("Encrypted:", C1, C2)

M2 = decrypt(C1, C2, private_key)
print("Decrypted:", M2)

```

**Output:**

```
"D:\New folder\New folder\.venv\Scripts\python.exe" "D:\New folder\New folder\Assign1 part 2.py"
Encrypted: (10, 6) (3, 1)
Decrypted: (6, 3)

Process finished with exit code 0
```

## Curve Parameters

The ECC system is built using a small finite field (mod  $p$ ).

$p = 17$  limits all calculations inside a finite range

$a = 2, b = 2$  define the elliptic curve equation

These parameters create the mathematical structure used for encryption.

## Point Addition

Combines two different points on the curve

Uses slope formula

Produces a new valid curve point

## Point Doubling

Special case when both points are same

Used in repeated multiplication

## Scalar Multiplication (Most Important Part)

`scalar_mult(k, P)`

Repeatedly adds a point to itself

Equivalent to:

$k \times P = P + P + P + \dots$

## **Encryption**

Random number  $k$

Base point  $G$

Public key

It produces:

$$C1 = k \times G$$

$$C2 = \text{Message} + (k \times \text{Public Key})$$

## **Decryption**

Uses private key to recreate shared secret

Removes encryption effect

Recovers original message point

## **Security Analysis**

### **Data Security**

ECC security is based on the Elliptic Curve Discrete Logarithm Problem (ECDLP).

Meaning:

Easy to compute forward (public key generation)

Extremely hard to reverse (finding private key)

### **Key Strength Advantage**

ECC gives same security as RSA but with smaller keys

Faster computation

Less memory usage

### **3.3 Tamper Resistance**

Any change in ciphertext breaks decryption

Invalid keys produce wrong output